

LLD Case Study 8: In-Memory API Rate Limiter

Target Patterns: Strategy Pattern, Encapsulation, State Synchronization
Key Engineering Focus: High-throughput rate-limiting algorithms: Token Bucket, Leaky Bucket, and Sliding Window Log with thread-safe client isolation.

1. Problem Statement & Functional Requirements

Design an API rate limiter capable of protecting backend microservices from Denial-of-Service (DoS) attacks, brute-force requests, and resource starvation.

Requirements:

- 1. Pluggable Algorithms (Strategy Pattern):** Easily interchange between **Token Bucket**, **Leaky Bucket**, and **Sliding Window Log**.
- 2. Client Isolation:** Maintain independent rate limits per Client ID / IP address.
- 3. Thread-Safety:** Atomic token consumption under high concurrent request volume.
- 4. Instant Evaluation:** Must execute in under 1 millisecond ($O(1)$ time complexity).

2. Algorithm Comparison & Trade-Offs

Algorithm	Memory Consumption	Burst Handling	Accuracy	Use Case
Token Bucket	$O(1)$ per client	Supports bursts up to bucket capacity	High	General API Gateways (Stripe, GitHub)
Leaky Bucket	$O(\text{Capacity})$ buffer	Enforces smooth, fixed outflow rate	High	Egress traffic shaping
Sliding Window Log	$O(N)$ requests in window	Accurate without boundary resets	100% Exact	Strict financial APIs

```
classDiagram
    class RateLimitingStrategy {
        <<interface>>
        +allowRequest(String clientId) bool
    }

    class TokenBucketLimiter {
        -int capacity
        -float refillRatePerSecond
        -Map~String, ClientBucket~ clientBuckets
        +allowRequest(String clientId) bool
    }

    class SlidingWindowLogLimiter {
        -int maxRequests
        -float windowSeconds
        -Map~String, List~float~ requestLogs
        +allowRequest(String clientId) bool
    }

    class RateLimiterService {
        -RateLimitingStrategy strategy
        +isAllowed(String clientId) bool
    }

    RateLimiterService o-- RateLimitingStrategy : Strategy
    RateLimitingStrategy <.. TokenBucketLimiter : Implements
    RateLimitingStrategy <.. SlidingWindowLogLimiter : Implements
```

3. Production-Grade Python Implementation

```
import time
import threading
from abc import ABC, abstractmethod
from typing import Dict, List
from collections import deque

# --- Strategy Interface ---
class RateLimitingStrategy(ABC):
    @abstractmethod
    def allow_request(self, client_id: str) → bool:
        pass

# --- Algorithm 1: Token Bucket ---
class TokenBucketLimiter(RateLimitingStrategy):
    class Bucket:
        def __init__(self, capacity: int, refill_rate: float):
            self.capacity = capacity
            self.refill_rate = refill_rate
            self.tokens = float(capacity)
            self.last_refill = time.monotonic()
            self.lock = threading.Lock()

        def consume(self) → bool:
            with self.lock:
                now = time.monotonic()
                elapsed = now - self.last_refill
                self.last_refill = now

                # Refill tokens proportionally to elapsed seconds
                self.tokens = min(float(self.capacity), self.tokens + (elapsed * self.refill_rate))

                if self.tokens ≥ 1.0:
                    self.tokens -= 1.0
                    return True
                return False

    def __init__(self, capacity: int, refill_rate_per_sec: float):
        self.capacity = capacity
        self.refill_rate = refill_rate_per_sec
```

```

        self.buckets: Dict[str, TokenBucketLimiter.Bucket] = {}
        self._global_lock = threading.Lock()

    def _get_bucket(self, client_id: str) → Bucket:
        with self._global_lock:
            if client_id not in self.buckets:
                self.buckets[client_id] = self.Bucket(self.capacity, self.refill_rate)
            return self.buckets[client_id]

    def allow_request(self, client_id: str) → bool:
        return self._get_bucket(client_id).consume()

# --- Algorithm 2: Sliding Window Log ---
class SlidingWindowLogLimiter(RateLimitingStrategy):
    class Log:
        def __init__(self):
            self.timestamps = deque()
            self.lock = threading.Lock()

    def __init__(self, max_requests: int, window_seconds: float):
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self.logs: Dict[str, SlidingWindowLogLimiter.Log] = {}
        self._global_lock = threading.Lock()

    def _get_log(self, client_id: str) → Log:
        with self._global_lock:
            if client_id not in self.logs:
                self.logs[client_id] = self.Log()
            return self.logs[client_id]

    def allow_request(self, client_id: str) → bool:
        client_log = self._get_log(client_id)
        now = time.monotonic()
        threshold = now - self.window_seconds

        with client_log.lock:
            # Evict timestamps outside active sliding window
            while client_log.timestamps and client_log.timestamps[0] ≤ threshold:
                client_log.timestamps.popleft()

            if len(client_log.timestamps) < self.max_requests:

```

```
        client_log.timestamps.append(now)
        return True
    return False

# --- Context Service ---
class RateLimiterService:
    def __init__(self, strategy: RateLimitingStrategy):
        self.strategy = strategy

    def process_request(self, client_id: str) → bool:
        allowed = self.strategy.allow_request(client_id)
        status = "ALLOWED" if allowed else "BLOCKED (429 Too Many Requests)"
        print(f"[RateLimiter] Client '{client_id}': {status}")
        return allowed

# --- Verification Driver ---
if __name__ == "__main__":
    # Test Token Bucket: Capacity = 3, Refill = 1 token/sec
    limiter = RateLimiterService(TokenBucketLimiter(capacity=3, refill_rate_per_sec=1.0))

    client = "client_ip_192.168.1.100"
    for i in range(5):
        limiter.process_request(client)

    print("\nSleeping 1.5 seconds for token refill ...")
    time.sleep(1.5)
    limiter.process_request(client)
```